Computer Security 2026 System Security

Roberto Guanciale

Memory bugs - effects

Access to Secret data

Corruption of program Data

Unexpected transfer of Control

Memory access Violation

Execution of code chosen by attacker

Binary Search

```
search(A, x, L, R):
    if L > R:
        return -1
    M = (L+R)/2
    if A[M] == x: return M
    if A[M] > x: return search(A, x, L, M-1)
    if A[M] < x: return search(A, x, M+1, R)
```

Row hammer

- side effect in dynamic random-access memory (DRAM)
  - memory cells interact electrically between themselves possibly discharging
- Hammering (reading multiple times) a memory line can flip a bit of another line
- Used in javascript for example to flip a bit of the JIT code, possibly a pointer
- Defense, ECC memories, physically separate memory on DRAM

Preventing Detecting Memory unsafety

Buffer overflow defenses

---

buffer overflows are widely exploited

- large amount of vulnerable code in use
- despite cause and countermeasures known

three defense approaches

- compile-time - harden new programs
- run-time - handle attacks on existing programs
- hardware-based defenses

Compile time Defenses: Language

---

use a modern high-level languages with strong typing

- you can not access untyped memory
- not vulnerable to buffer overflow

implementations combine compile-time restrictions, run-time checks, and memory management.

rust uses ownership and borrowing to enforce many lifetime rules without garbage collection.

- do have cost in performance, memory usage, and timing predictability
- hardware access and foreign interfaces may require trusted **unsafe** code
- guarantees depend on the correctness of unsafe components, libraries, compilers, and runtimes.

Compile time Defenses: safe coding

---

if using potentially unsafe languages e.g. C

- Check buffer capacity before every read or write.
- Validate lengths and check for integer overflow in size calculations.
- Handle allocation failures before using the returned pointer.
- Define ownership and ensure that accesses remain within an object’s lifetime.
- For strings, account for the terminating null byte and handle truncation explicitly.
- use safer standard library variants
  - new functions, e.g. strncpy()
  - strncpy() can leave the destination without a terminating null byte.

Static analysis and formal verification

- Static analysis examines code without executing it and can identify potential defects.
- Formal verification proves specified properties using mathematical models.
- A proof establishes a guarantee
  .

Expensive: ~2000$ (2010) ~500$ (2020) per line of code

Verified execution platforms

- isolation/separation kernels (seL4)
- software fault isolation (ARMor)
- tools such as Frama-C

Static analysis and formal verification

- Static analysis examines code without executing it and can identify potential defects.
- Formal verification proves specified properties using mathematical models.
- A proof establishes a guarantee under explicit assumptions about the program and its environment.

Expensive: ~2000$ (2010) ~500$ (2020) per line of code

Verified execution platforms

- isolation/separation kernels (seL4)
- software fault isolation (ARMor)
- tools such as Frama-C

Dynamic Verification (aka Testing)

- Testing executes a program with selected inputs.
- Fuzzing generates or mutates inputs to explore program behavior.
- Symbolic execution represents inputs symbolically and solves path constraints to generate test cases.
- Monitor program execution to identify bugs
  - Memcheck instruments the program to track
    - addressability: whether a memory location may be accessed.
    - definedness: whether a value has been initialized.

Dynamic Verification (aka Testing)

- Testing executes a program with selected inputs.
- Fuzzing generates or mutates inputs to explore program behavior.
- Symbolic execution represents inputs symbolically and solves path constraints to generate test cases.
- Monitor program execution to identify bugs
  - Memcheck instruments the program to track
    - addressability: whether a memory location may be accessed.
    - definedness: whether a value has been initialized.
- Testing can reveal defects in exercised executions. Passing tests does not prove the absence of defects.

Background: HW based Memory protection

- Memory Management Unit
  - Page tables stored in memory
  - Configured by OS
  - Maps Virtual Addresses to Physical Addresses and access permissions
- Memory Protection Unit
  - Configuration in special registers
  - Configured by OS
  - Maps physical memory regions to access permissions

Run time defense: guard pages

- configured in Memory Management Unit as illegal addresses
- any access aborts process

can be placed between

- stack frames and heap buffers
- between critical regions of memory
  - between buffers of different network sessions
  - between network buffers and TLS certificates/JIT memory

Example of Buffer overread

```
void main(int argc, char **argv) {
    char name[8];
    char pwd[8];
    int i, n = 0;

    strcpy(pwd, "pwd0");
    strcpy(name, argv[1]);
    n = atoi(argv[2]);

    printf("Echo ");
    for (i = 0; i < n; i++) {
        printf("%c", name[i]);
    }
    printf("\n");
}
```

```
./main roberto 0
> echo
./main roberto 1
> echo r
./main roberto 7
> echo roberto
./main roberto 16
echo roberto������
./main roberto 32
echo roberto�������pwd0����
```

Other memory safety problems

Dangling pointers: do not point to a valid object of the appropriate type

Causes:

- wrong dynamic cast of pointers

```
Cat {int x; int y}

Dog {int x}
```

```
Dog * x = malloc(sizeof(Dog));

add(list, x);

...

void * y = get(list,0);

miaow((Cat *) y);
```

Fat pointers

```
Buffer {
    random key
    data
}
Pointer {
    key
    base
    bound
    pointer
}
```

Allocation:

- Generate a random key
- Return fat-pointer

Pointers arithmetic

- Check bounds

Dereference

- Check base key = pointer key

Free

- New random key for buffer

ARM Memory Tagging Extension

- Each 16-byte memory **granule** has a 4-bit allocation tag
- A pointer's logical tag occupies address bits 59-56.
- Memory Tag DB
  - 1 granule => 1 tag
  - Raw tag storage is 4 bits per 16 bytes: 3.125%.
  - For 16 GiB of tagged memory, 512 MiB of tag storage

MTE-aware malloc:

- Generate a random tag for the buffer, update the memory tag DB entries for **each** granule, return a tagged pointer

Memory Access (in HW)

- Check pointer tag = granule tag

MTE-aware free:

- Check pointer tag = granule tag, generate a new random tag for the buffer's granules

Other memory safety problems

Dangling pointers: do not point to a valid object of the appropriate type

Causes:

- wrong dynamic cast of pointers

```
Cat {int x; int y}

Dog {int x}
```

```
Dog * x = malloc(sizeof(Dog));

add(list, x);

...

void * y = get(list,0);

miaow((Cat *) y);
```

Intra object overflow

```
struct User {
    bool root;
    char name[16]
}
```

```
gets(user.name)
If (user.root) {
    ...
}
```

Mitigating Memory unsafety

i.e., preserving some properties of vulnerable code

Control flow integrity

Prevent or detect alteration of the control flow due to

- Modification of return pointer
- Modification of a function pointer

Compile time Defenses: Stack protection

---

add entry and exit code to check stack for signs of corruption

- use random (different for every execution) canary
- e.g. GCC -fstack-protector (not all functions)
- Canaries were used in coal mines to detect the presence of carbon monoxide



Compile time Defenses: Stack protection

---

add entry and exit code to check stack for signs of corruption

- use random (different for every execution) canary
- e.g. GCC -fstack-protector (not all functions)

check for overwrite between local variables and saved frame pointer and return address

- abort program if change found
- issues: recompilation, debugger support

Compile time Defenses: Stack protection

---

add entry and exit code to check stack for signs of corruption

- use random (different for every execution) canary

Stack:

Return ptr
Frame ptr
Local var1
Local var2
Param 1
Param 2

Compile time Defenses: Stack protection

---

add entry and exit code to check stack for signs of corruption

- use random (different for every execution) canary

Stack:

```
Return ptr
Frame ptr
1243152311653    // Canary
Local var1
Local var2
Param 1
Param 2
```

Example of Buffer overread

```
void main(int argc, char **argv) {
    char name[8];
    char pwd[8];
    int i, n = 0;

    strcpy(pwd, "pwd0");
    strcpy(name, argv[1]);
    n = atoi(argv[2]);

    printf("Echo ");
    for (i = 0; i < n; i++) {
        printf("%c", name[i]);
    }
    printf("\n");
}
```

```
./main roberto 0
> echo
./main roberto 1
> echo r
./main roberto 7
> echo roberto
./main roberto 16
echo roberto������
./main roberto 32
echo roberto�������pwd0����
```

Buffer Overflow example

```c
int main(int argc, char **argv) {
    char name[8];
    char pwd[8];

    printf("Start\n");

    strcpy(pwd, "pwd0");
    strcpy(name, argv[1]);

    printf("Hello %s\n", name);
    printf("PWD %s\n", pwd);
    printf("Pointer %p\n", name);
    printf("Pointer %p\n", pwd);

    if (strcmp(pwd, argv[2])) {
        printf("non authorized\n");
    } else {
        printf("authorized\n");
    }

    printf("End\n");
    return 0;
}
```

Address	Content	Variable
0x7fffffffdcd0	1234567890123456	name
0x7fffffffdce0	hello\0	pwd

Can I login with a wrong password?

Can I modify the password?

./main 1234567890123456hello hello

Control Flow Subversion: Return attack

```c
void hello(char *msg) {
    char buffer[16];

    printf("&msg adr \t%p\n", &msg);
    printf("msg adr \t%p\n", msg);
    printf("buffer adr \t%p\n", buffer);

    printf("enter the message for %s: \n", msg);
    printf("adr \t\t%p\n", *(void **)(buffer + 16));
    printf("adr \t\t%p\n", *((void **)(buffer + 24)));

    gets(buffer);

    printf("message for %s is %s\n", msg, buffer);
    printf("adr \t\t%p\n", *(void **)(buffer + 16));
    printf("adr \t\t%p\n", *(void **)(buffer + 24));

    return;
}

int main(int argc, char **argv) {
    char mainTag[16] = "Roberto";

    printf("main adr \t%p\n", &main);
    printf("hello adr \t%p\n", &hello);
    printf("mainTag adr \t%p\n", mainTag);

    hello(mainTag);

    return 0;
}
```

```
>./main

main adr                0x40068c
hello adr            0x400586
mainTag adr      0x7fffffffdd00

&msg adr    0x7fffffffdcc8
msg adr          0x7fffffffdd00
buffer adr       0x7fffffffdcd0

> Enter the message for Roberto
adr              0x7fffffffdd10
adr                      0x4006fb

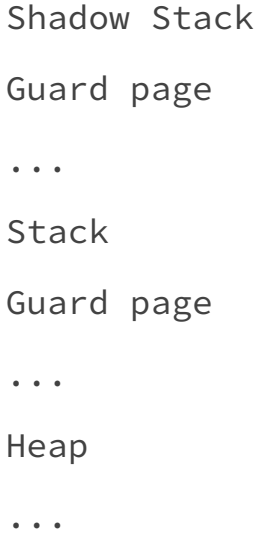
12345678901234 56abcdabcd12345678
> Message for Roberto is Hello
adr              0x6463626164636261
adr              0x400586

#!/usr/bin/python
sys.stdout.buffer.write(b'a' * 24)
sys.stdout.buffer.write(
     (0x400586).to_bytes(8, 'little'))
```

```
void f(char * input1, int input2) {
    int * x = malloc(8);
    char buffer[16];
    strcpy(buffer, input1);
    *x = input2;
    return;
}
```

Compile time Defenses: Stack protection

- Store return addresses in a separate shadow stack.
- On return, compare against or restore the protected return address.
- Protect the shadow stack against writes from vulnerable program code.
- Shadow stacks protect returns. They do not protect arbitrary function pointers or application data.



```
void f(char * input1, int input2) {
    int * x = malloc(8);
    char buffer[16];
    strcpy(buffer, input1);
    *x = input2;
    return;
}
```

Run time Defenses: Target address encryption

- Indirect jumps (e.g. jumps to non-constants) are necessary
  - Function return
  - Callbacks, Virtual methods, Exception handling
- Encrypt the addresses
  - Notice xor is not a good way of doing encryption

```
int (*func1)(int) = double;

func1(15);


====>


int (*func1)(int) = double ^ key;

(func1 ^ key)(15);
```

Run time Defenses: Control Flow Attestation

- Provide authenticated evidence of an execution control-flow path for a verifier to check
- Alter the system to keep track of PC updates
- To update the measurement:
  - HW modifications
  - Static analysis of no indirect jmp, all indirect jumps are made by a trusted SW via a trampoline
- To keep track of the measurements
  - Only hash
  - Isolated memory

```
int (*func1)(int) = double;

func1(15);

====>

int (*func1)(int) = double;

invoke(func1, 15);

invoke(f,args) {

     log(f);

     (f)(args);

}
```

Code integrity

- Prevention, detection, mitigation of code injection
- Due to a buffer controlled by the attacker (e.g. where a network packet is stored) being executed
- Due to existing code being overwritten

Code integrity: Executable Address Space Protection

- use virtual memory support to make some regions of memory non-executable
  - e.g. stack, heap, global data need HW support in MMU
- long existed on SPARC / Solaris systems
- recent on x86/ARM Linux/Unix/Windows systems

STACK	R/W
HEAP	R/W
Global Data	R/W
Code	R/X

Code integrity: Executable Address Space Protection W⊕X

- use virtual memory support to make some regions of memory non-executable
  - e.g. stack, heap, global data need HW support in MMU
- long existed on SPARC / Solaris systems
- recent on x86/ARM Linux/Unix/Windows systems
- issues: support for executable stack/heap code
  - needed for JIT (e.g. Java) or nested functions
  - need special provisions
    - mprotect(ptr, size, (PROT_READ | PROT_EXEC);
- Attacker can
  - Inject payload
  - Corrupt control flow to invoke mprotect (see ROP)
  - Execute the payload

Code integrity: code signature

- Before executing a program, check that it satisfies the platform's trust policy, usually by hashing its binary

  Options:

- Keep protected database of valid digests, check that SHA(program) in DB
- Use program with certificate
  - signature = Enc_{MSsk}(SHA(program))
  - Check Dec_{MSpu}(signature) = SHA(program)
  - Chain of certificates can be used to prove authorship and integrity
- Problem
  - Pages are usually loaded dynamically, therefore expensive for entire program

Code integrity: run-time code signature

- Enforce W⊕X policy
- Check signature whenever a page becomes executable (i.e. mprotect)

  Options for signature checks

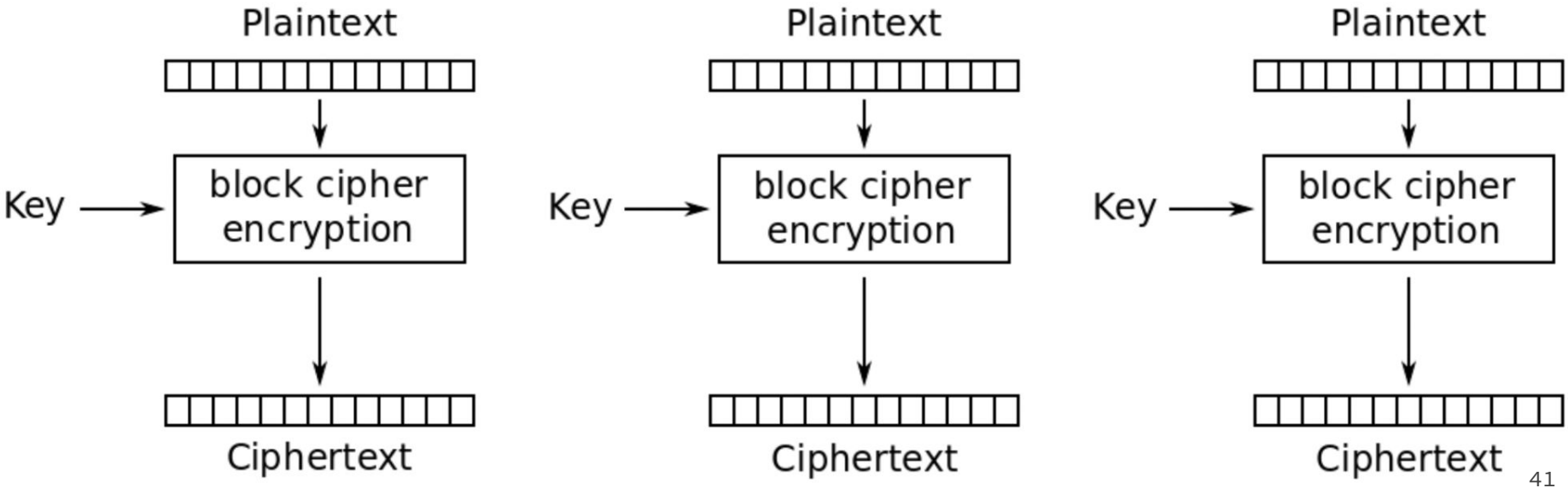
- Keep protected database of valid digests, check that SHA(page) in DB
- Use program with certificate
  - signature = Enc_{MSsk}(SHA(page))
  - Check Dec_{MSpu}(signature) = SHA(page)
  - Chain of certificates can be used to prove authorship and integrity
- Keep a database of binary fragments of well known malwares, check that Intersect(page, db) = empty
- JIT compilation requires a controlled policy for generating and enabling executable code.
  - See MProtect

Related topic: Disk Encryption

Would you use symmetric or asymmetric encryption?

Disk Encryption

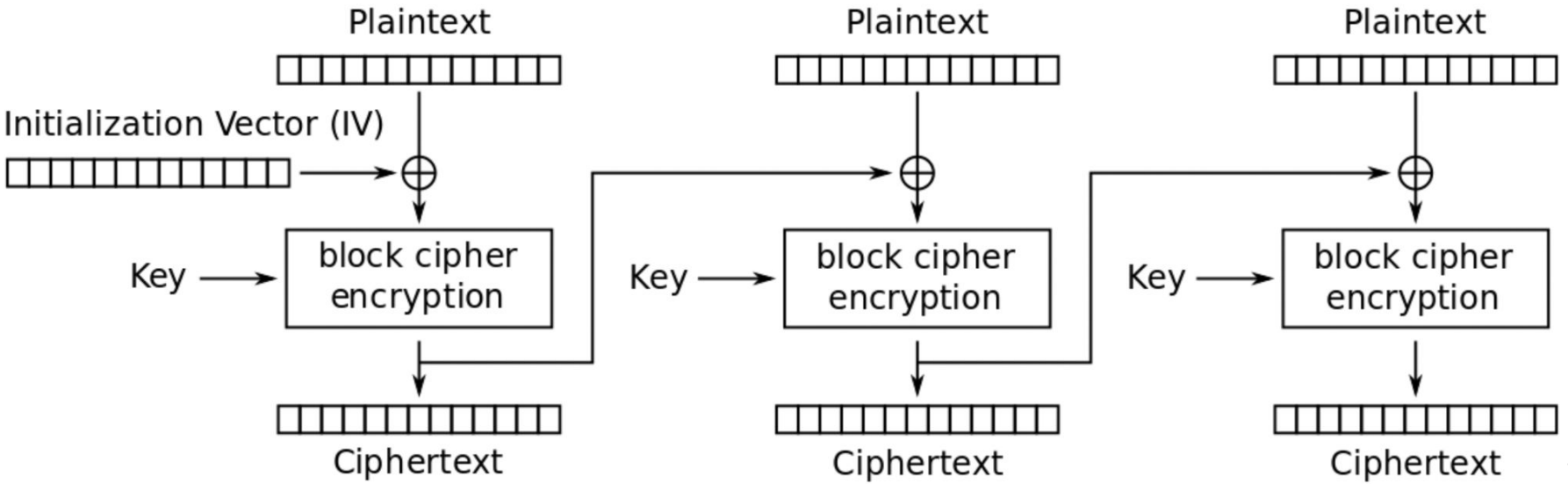
Is it secure to use AES ECB (Electronic codebook)?



Disk Encryption

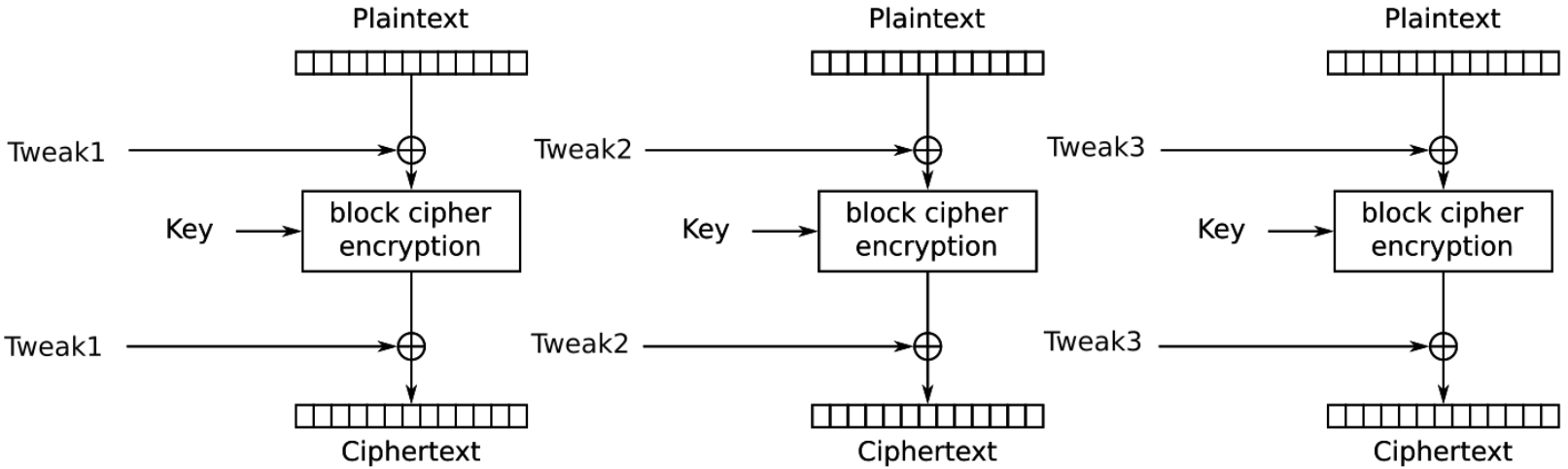
Is it secure to use AES ECB (Electronic codebook)?

Can we use Cipher Block Chaining?



Disk Encryption

Tweakable Encryption (For integrity we also need a MAC)



Xor Encrypt Xor (XEX) mode encryption

Instruction Set Randomization

Make every Process's CPU unique

If the attacker does not know the target ISA, it is difficult to produce injectable code

Instruction Set Randomization

Make every Process's CPU unique

If the attacker does not know the target ISA, it is difficult to produce injectable code

Code is encrypted in memory:

M[pc] = Encrypt(instruction)

Execution Of Instruction:

fetch M[pc] = c

i = Decrypt(c)

execute (i)

If attacker injects c’ they cannot predict how c’ will be decrypted (assuming attacker does not know key)

Decryption requirements

1. Cheap
   Symmetric block ciphers
2. Preserve instruction length
   No MAC
3. Support random accesses
   No Output feedback (OFB)
4. Use Tweakable Encryption

What each defense guarantees

Defense	Guarantee	Key assumptions	Remaining attacks
Memory-safe language	Prevents invalid memory accesses in safe code	Sound implementation and trusted unsafe interfaces	Logic errors, side channels, unsafe-code bugs
Stack canary	Detects changes to the canary before a protected return	Canary remains secret; overwrite crosses it; check executes	Canary disclosure, writes that bypass it, data corruption
W⊕X	Prevents writable and executable mappings of the same memory under the enforced policy.	Correct page permissions and control of writable aliases	ROP, data-only attacks, abuse of permission changes
ASLR	Makes target addresses harder to predict	Sufficient randomness and no useful address disclosure	Information leaks, guessing, relative overwrites
CFI / shadow stack	Restricts indirect targets / protects return addresses	Complete enforcement and protected metadata	Abuse of allowed targets, data-only attacks
Isolation / sandbox	Restricts a compromised component's access	Correct enforcement and limited permissions	Abuse of permitted operations, interface bugs, sandbox escape